

面向超大规模教育场景的Agent系统架构演进与工程实践

演讲人：王搂

科大讯飞 / 高级系统架构师

AiCon

全球人工智能开发与应用大会

目录

问题与挑战：教育超复杂场景下AI系统面临的挑战分析

WISH平台的核心定位：Agent时代的运行时底座

引擎托管与治理：如何把数百个引擎纳入统一框架

大模型时代的弹性底座：突破单云容量边界的跨云调度体系

演进展望：从确定性的运行时底座走向动态的、可自治的Agent基础设施

01 问题与挑战

教育超复杂场景下AI系统面临的挑战分析

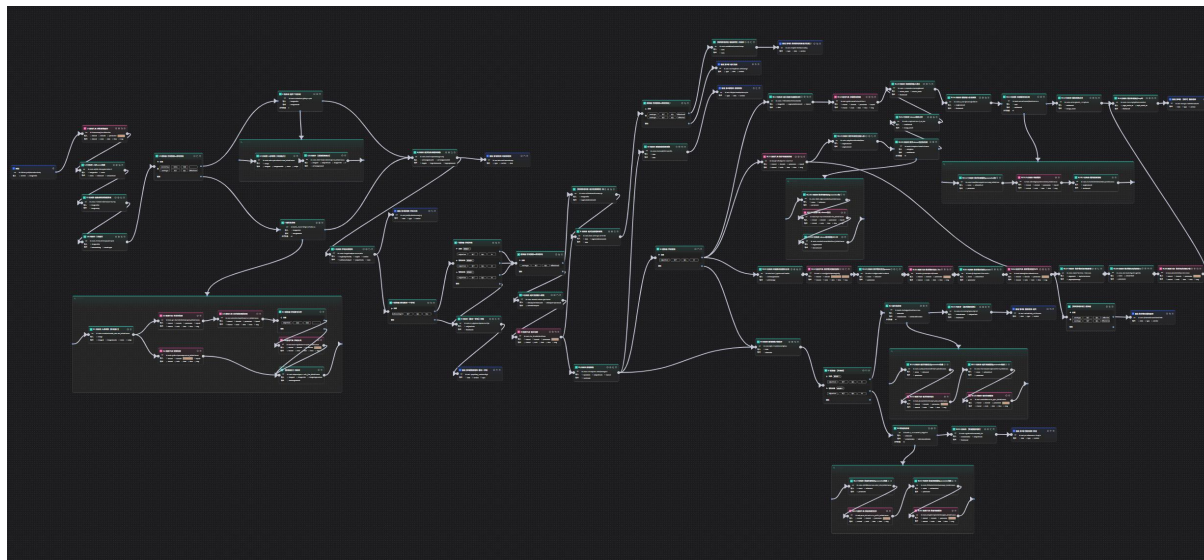
教育AI应用开发面临的新挑战

“单能力、细窄串行链路、单轮、半双工、短会话”



局面发生了深刻的变化... ..

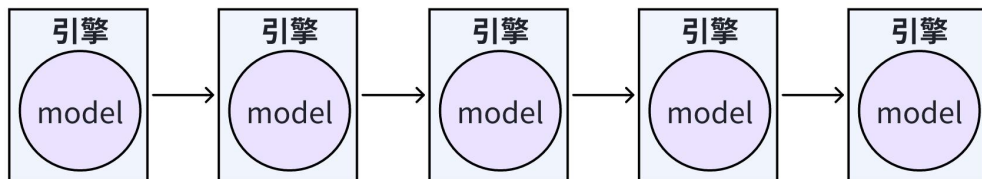
“多能力、长肥复杂DAG、多轮、全双工、长会话”



- 针对复杂需求，大模型的能力依然无法将意图理解和行动规划做到鲁棒好用的水平，需要 AI 方案架构师设计施工图（一个复杂 DAG，组合多个AI模型），并在上线后收集数据不停迭代方能满足教育场景的高要求
- 在辅导答疑、智能批改等教育核心场景中，涉及几十上百个引擎，单次任务的持续时间可能长达数分钟，期间涉及多轮交互，交互形式实时与异步同时存在，使开发、调试、发布、运维全部进入高复杂度区间
- 复杂AI应用的施工图已演变成超复杂DAG，静态的链路流程应对教育场景对效果和性能的要求已有些捉襟见肘

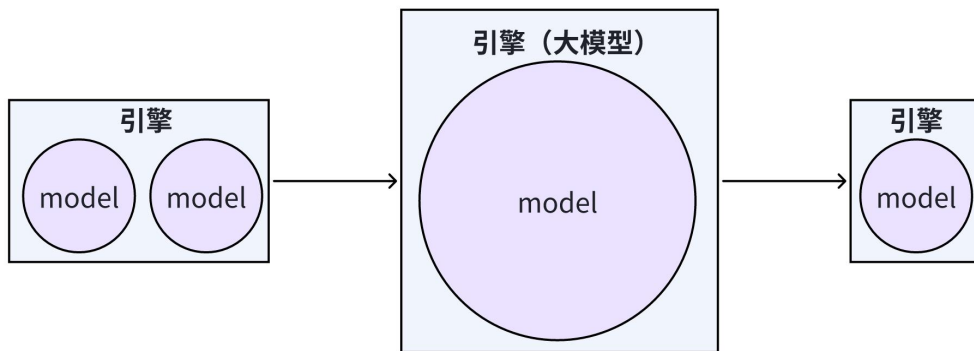
教育AI引擎开发面临的新挑战

形态一：引擎单机单卡可容纳
一个引擎一个模型

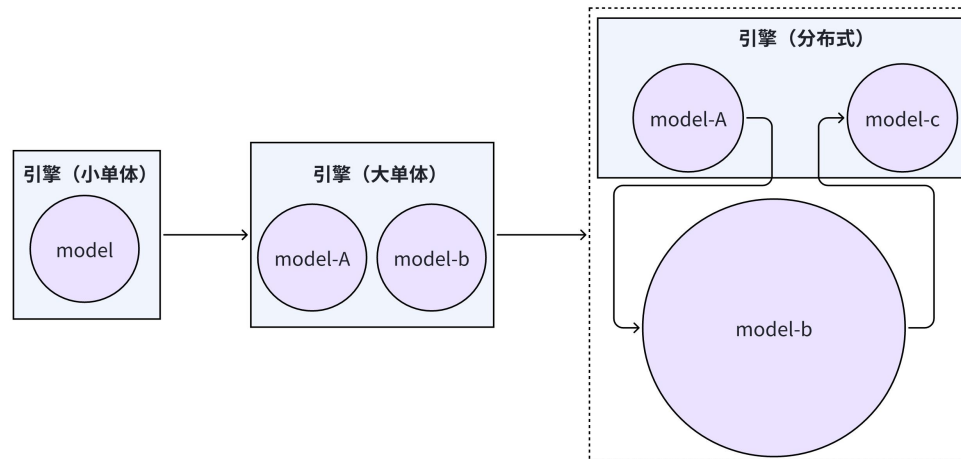


形态二：引擎单机多卡可容纳

一个引擎多个模型 or 一个引擎一个大模型



形态三：引擎多机多才能容纳

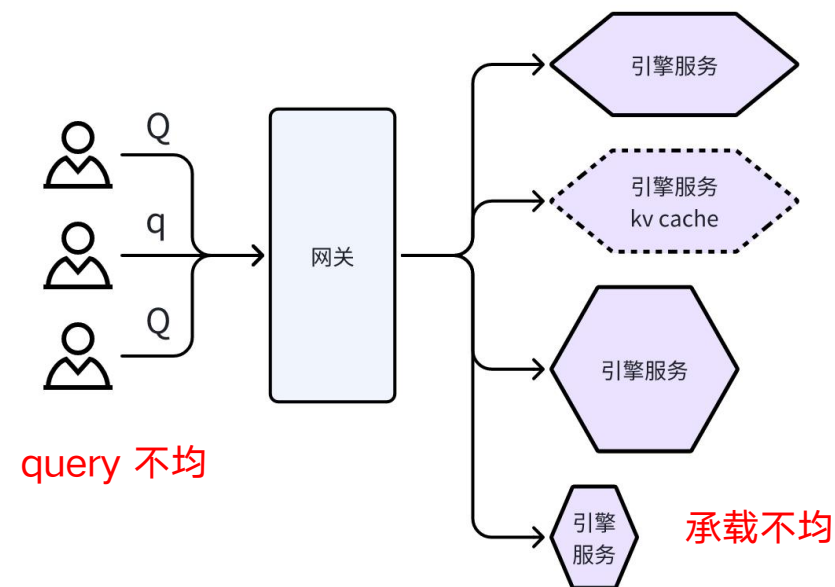
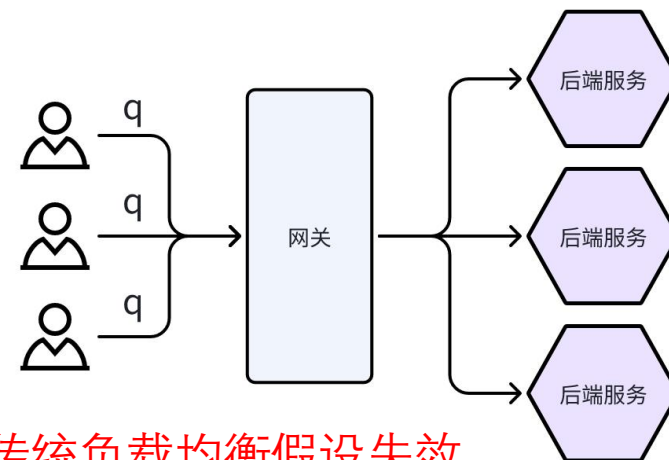


- 大尺寸模型的高额落地成本，迫使企业不断优化模型的部署和推理成本
- 模型引擎崩溃卡死频发，迫切需要有效手段解决快速保留故障现场以及避免连锁反应造成雪崩等问题

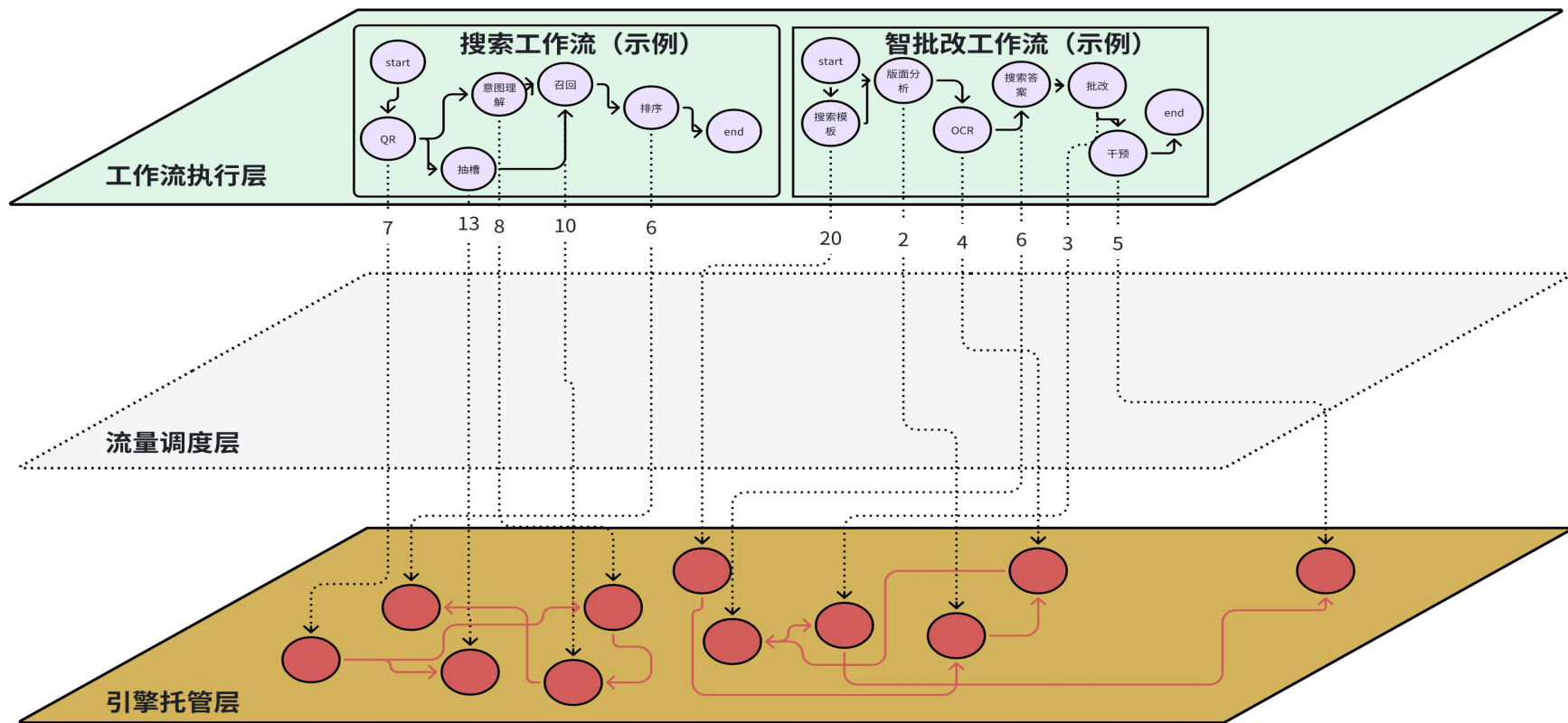
AI系统负载均衡面临的新挑战

传统负载均衡成立的两个基本前提：

- query大小均衡
 - 后端承载能力均衡
-
- 引擎数量众多，数百个引擎在生产环境运行，统一管理、统一调度迫在眉睫
 - 引擎的输入输出差距巨大，可能是数十M的图片，可能是一段数分钟的音视频，也可能是一段数M的文本
 - 引擎的处理能力和处理时长差异极大，可能可以同时处理几百个，也可能同时只能处理几个，处理时间更是从几毫秒到几分钟不等
 - AI引擎专属调度策略的诉求，需要考虑诸如kv cache命中等大模型场景专属的问题
 - AI系统引擎侧扇出情况严重，存在百倍放大的场景

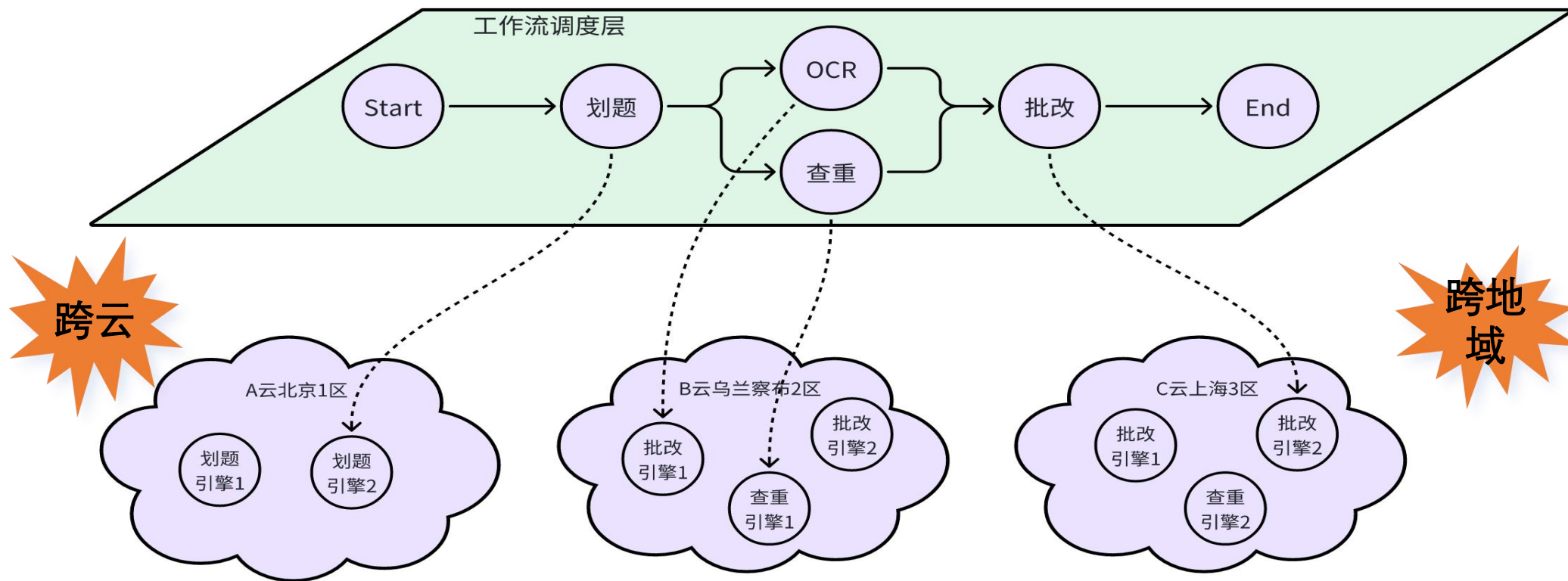


AI系统运维面临的新挑战1/2



- AI 系统变为“多层网络”，任意节点缺一不可，不同节点配比不同
- 系统的可观测和问题定位难度飙升，如何在上千节点、海量数据、异步系统中快速定位问题
- AI引擎对于资源的要求不再一视同仁，对于计算资源的要求千差万别

AI系统运维面临的新挑战2/2



- 公有云计算资源受限，可能需要**跨地域**、**跨公有云**调度资源才能满足资源要求
- 模型的巨大尺寸对AI引擎的快速拉起提出了超高的要求，跨云拉取上百G模型的长耗时（可达小时级），对于快速扩容的影响极为不利

小结

AI应用开发

多能力串联的复杂施工图，全双工的多轮长会话，固定链路流程导致DAG超复杂

AI引擎开发

引擎形态多样，多引擎多实例组合，大模型引擎PD分离

AI系统负载均衡

传统负载均衡假设失效，引擎数量众多，引擎负载特征差异大，大模型对kv cache亲和要求

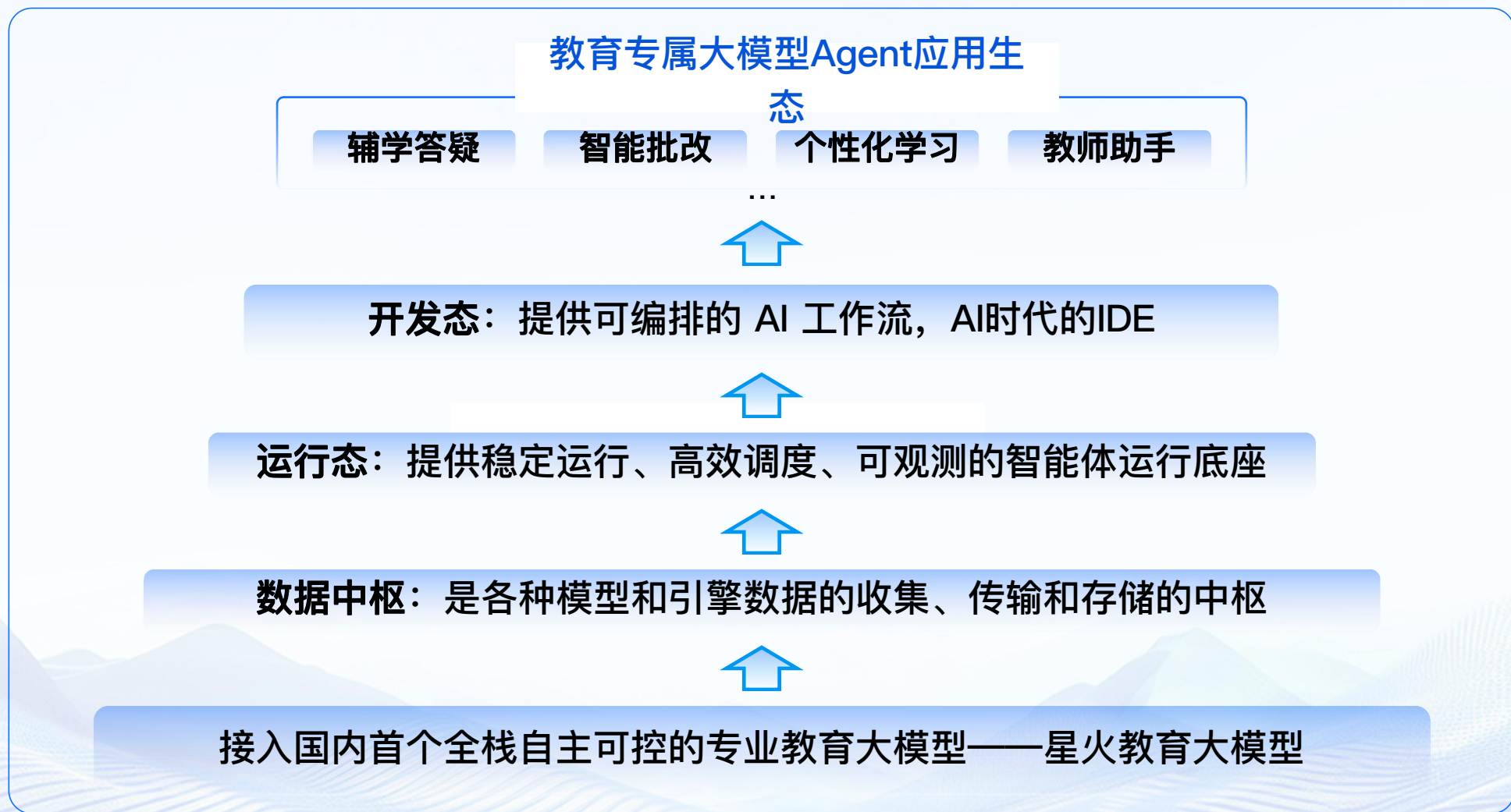
AI系统基础设施调度和运维

AI系统变成多层网络，计算资源差异大，跨地域跨公有云调度，模型尺寸超大，资源成本压力大

02 WISH平台的核心定位

Agent时代的运行时底座

讯飞WISH平台定位



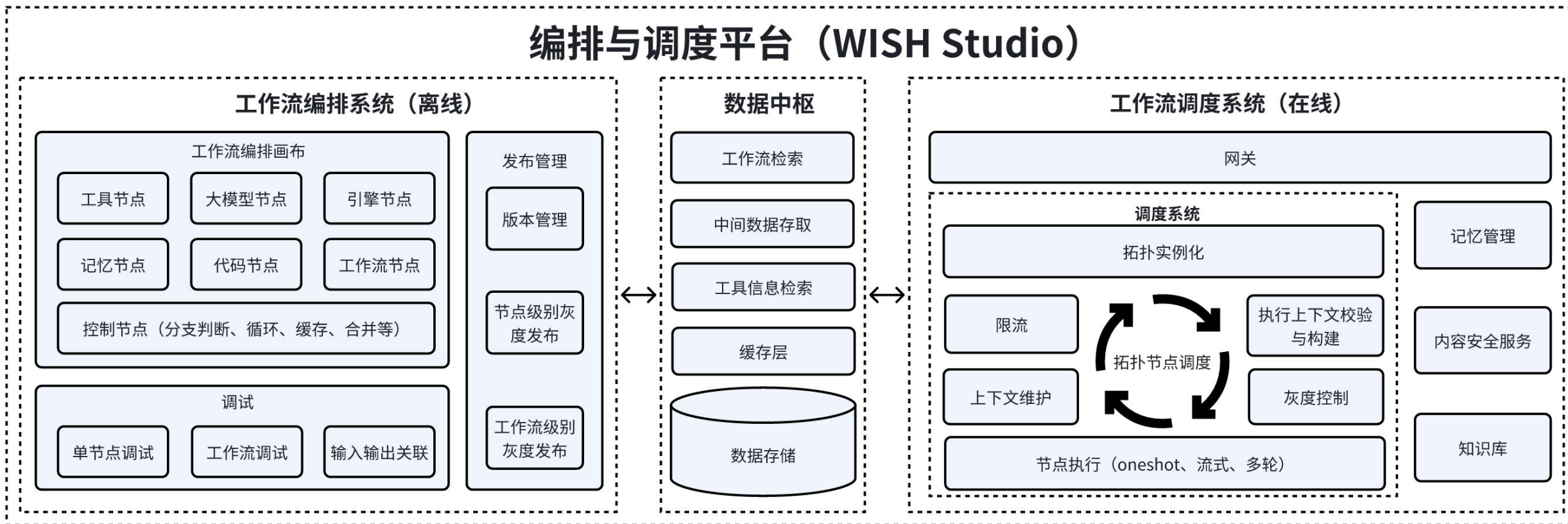
■ 对标编程语言和IDE的设计思路

WISH (Workflow Integration & Scheduling Hub) ， 对标传统 IDE 与编程语言， 专注教育复杂 AI 方案的研发与运行

- 遵从程序员开发习惯， 无缝从传统 IDE 迁移到WISH平台， 方便开发与测试人员研发测试复杂 AI 应用
- 高效编排拖拽， 画布应对95%的问题， Faas 解决剩下一公里
- 集开发、调试、版本管理、发布于一体

编程语言	WISH Studio
变量	会话变量、系统变量、节点变量
分支判断 (if-else/switch-case)	选择器节点， 分支合并节点
循环结构 (for/while/do-while)	循环节点
函数	子 workflow 节点
数据类型	整型， 字符串 ， 布尔型， 结构体， 列表
异常处理 (try-catch-finally/throw)	节点异常处理
并发/阻塞	批处理节点
标准库与三方库 (import, include)	工具节点、大模型节点
传统 IDE	WISH Studio
代码编辑	编排画布、代码编辑器
语言智能 (language intelligence)	自动填充、类型校验
调试能力	工作流调试
项目管理	工具管理、工作流管理
版本控制	工作流版本管理、工具版本管理
终端	控制台日志输出
插件生态	节点自定义 (暂不支持)
代码生成	接入大模型生成 prompt、code

WISH平台整体架构



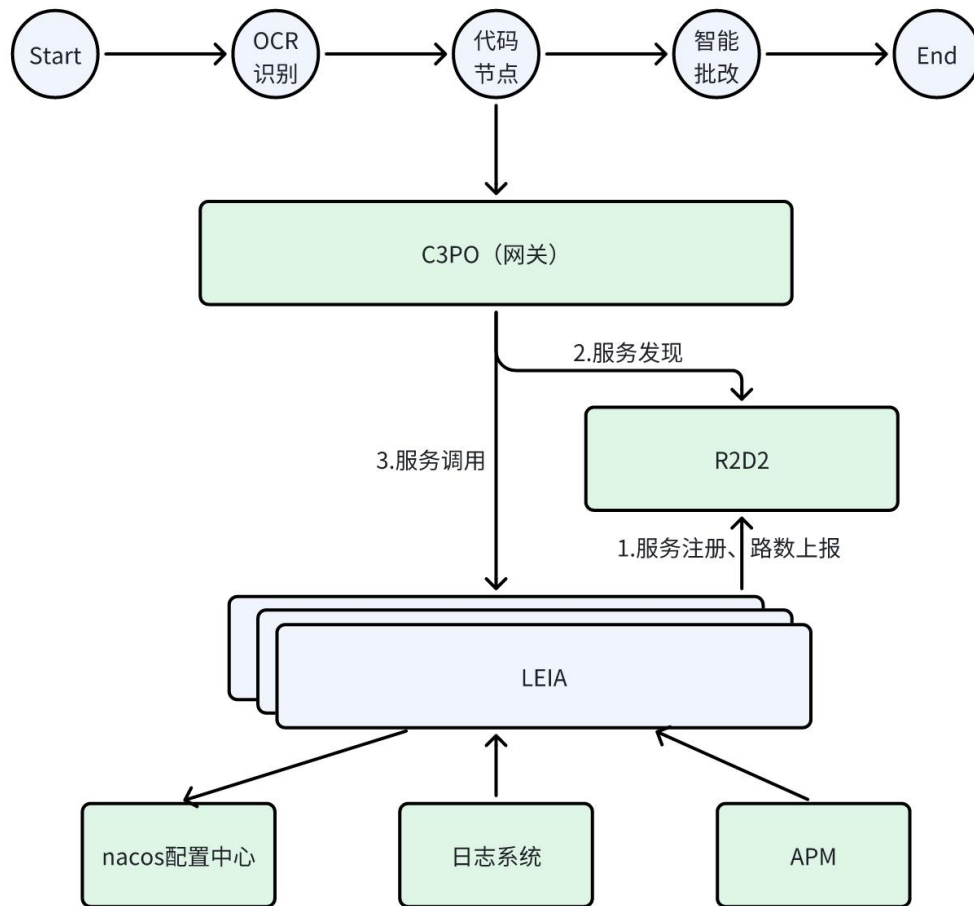
FaaS设计与落地

将代码执行从调度器解耦

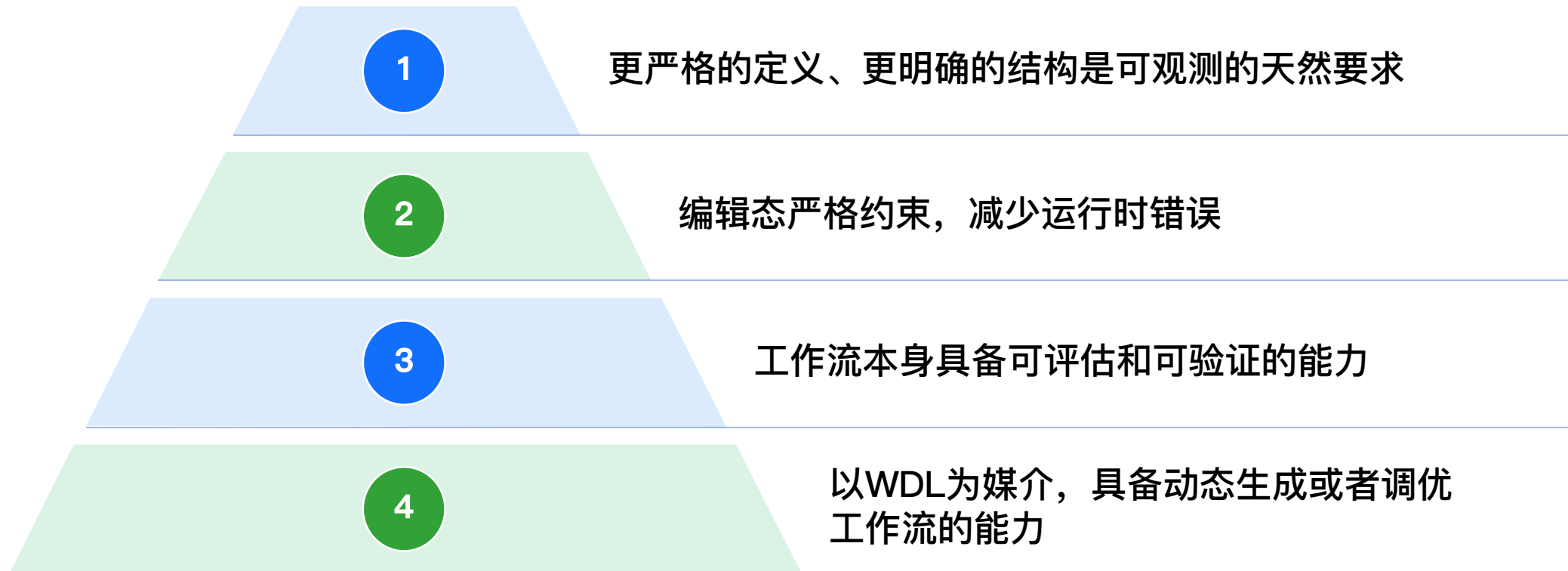
运行时引擎化，基于授权路数控制确保不过载

命名空间隔离降低串扰风险

预编译加缓存机制降低执行成本

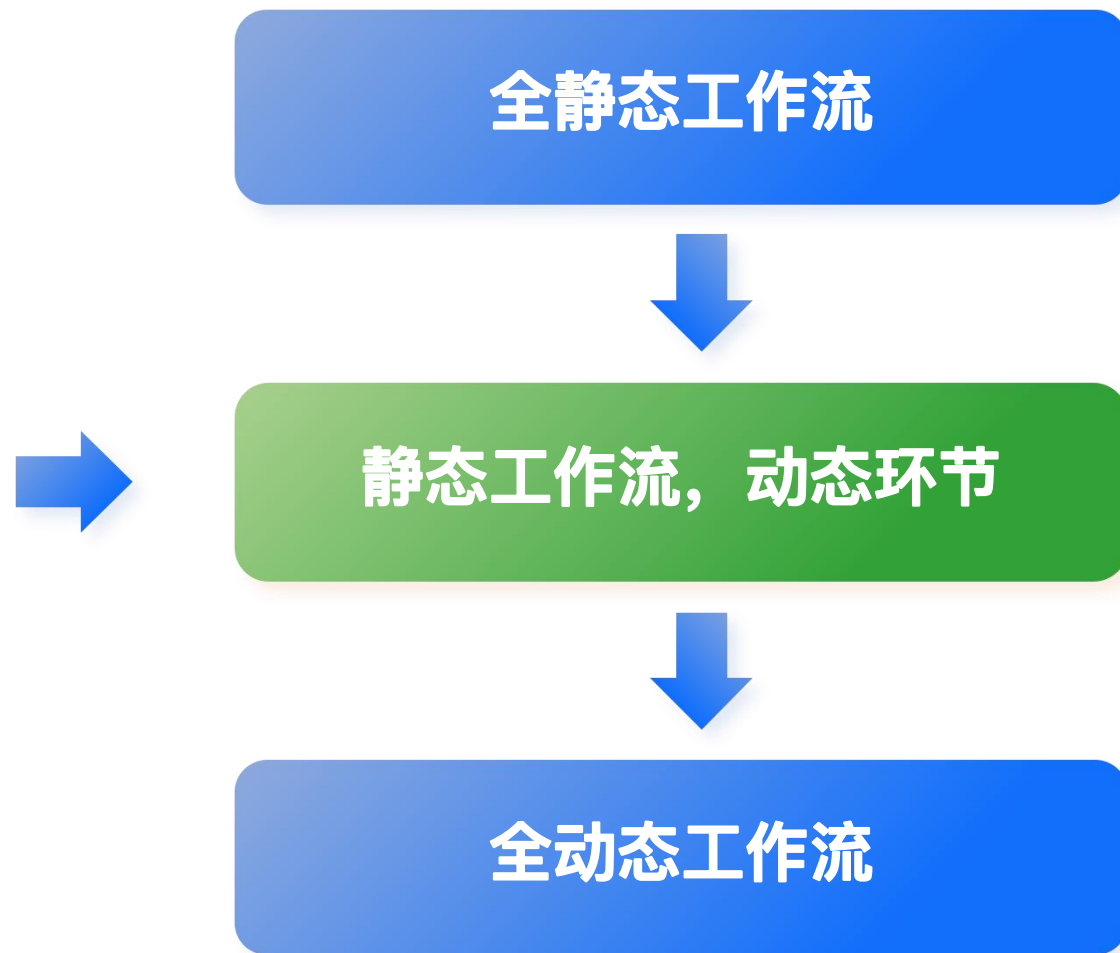


■ WDL(Workflow Description Language)



■ workflows形态的跃迁

- **输入不确定**：用户问题、上下文、文件、工具结果都可能动态变化，流程很难在设计时完全穷举。
- **执行路径不固定**：同一个任务可能因为模型判断、工具返回、异常状态走向不同分支。
- **中间状态越来越重要**：系统需要根据执行结果持续调整下一步，而不是一次性跑完整条链路。



小结



对标编程语言和 IDE，提供稳定运行时以及可观测、可循环的数据收集平台

平台定位



通过复杂DAG施工图，让复杂方案流程编排和调试可视化、简单化

DAG设计



FaaS让代码动作可隔离、可审计、可管控

FaaS



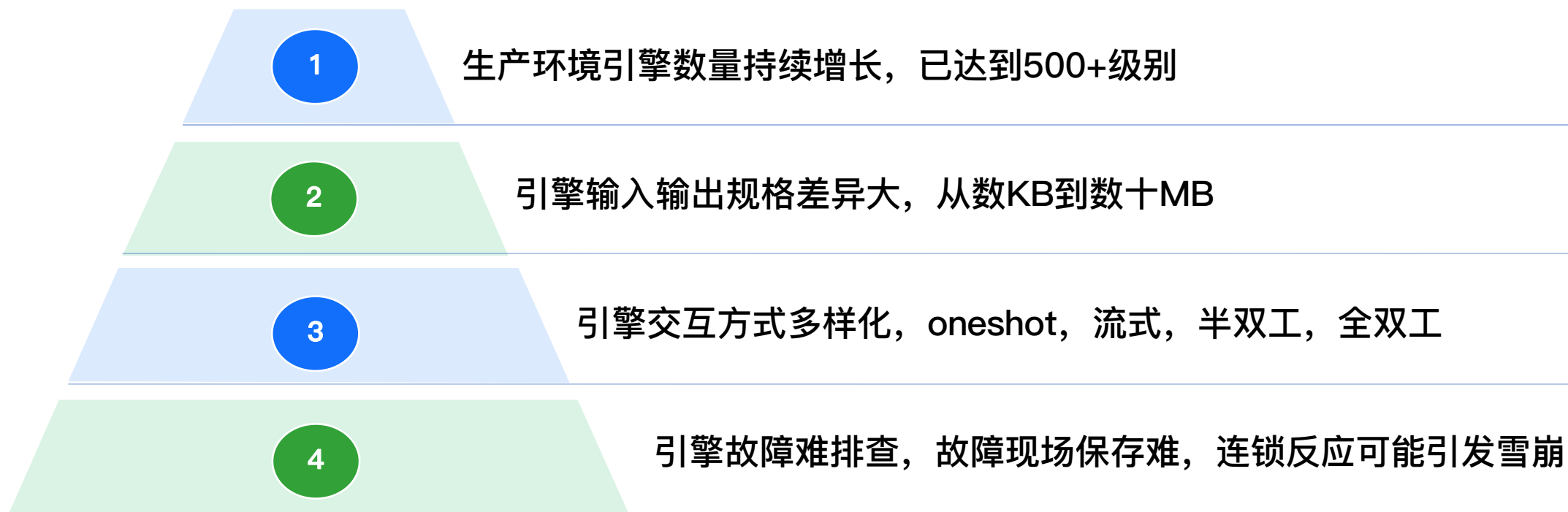
有效应对无法枚举，路径不固定的情况，避免工作流变成极端复杂的DAG

动态工作流

引擎托管与治理

如何把数百个引擎纳入统一框架

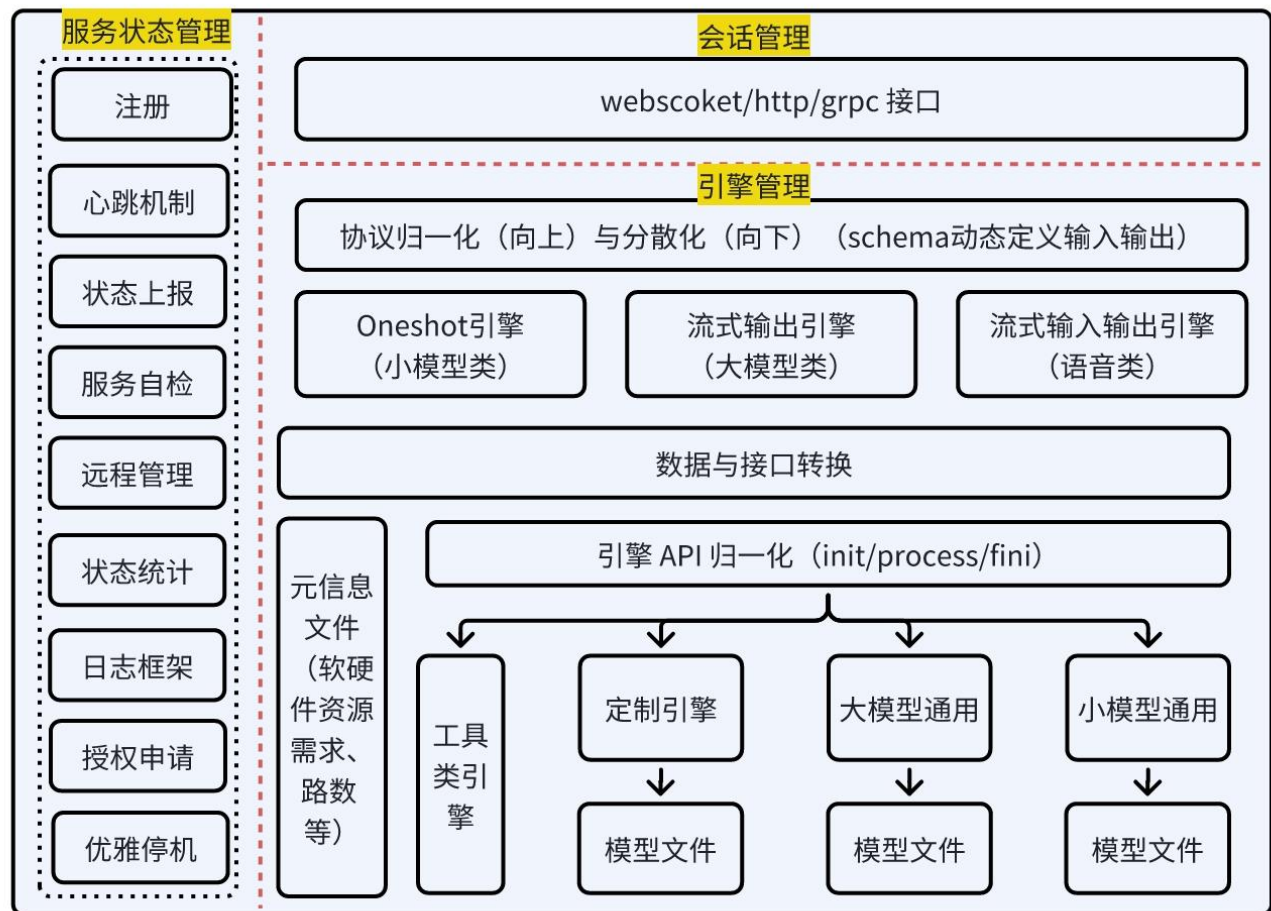
引擎托管的问题分析



统一引擎服务化框架

- **覆盖全**：oneshot与流式两种交互模式，传统小模型和LLM模型全覆盖
- **自动化**：一次编写无限次托管，无需任何额外开发工作
- **协议兼容**：灵活定义引擎输入输出schema协议，自动化生成入口协议，自动化校验
- **终身可追溯**：随包“户口本”+ 三层命名规范，彻底解决引擎管理混乱问题
- **启动自检，崩溃处理**：引擎启动一系列自检通过才对外服务，引擎挂掉minidump自动收集与分析解决崩溃现场收集难的困境，也避免了现场信息过于臃肿，有毒请求过滤避免出现雪崩问题

引擎服务化框架

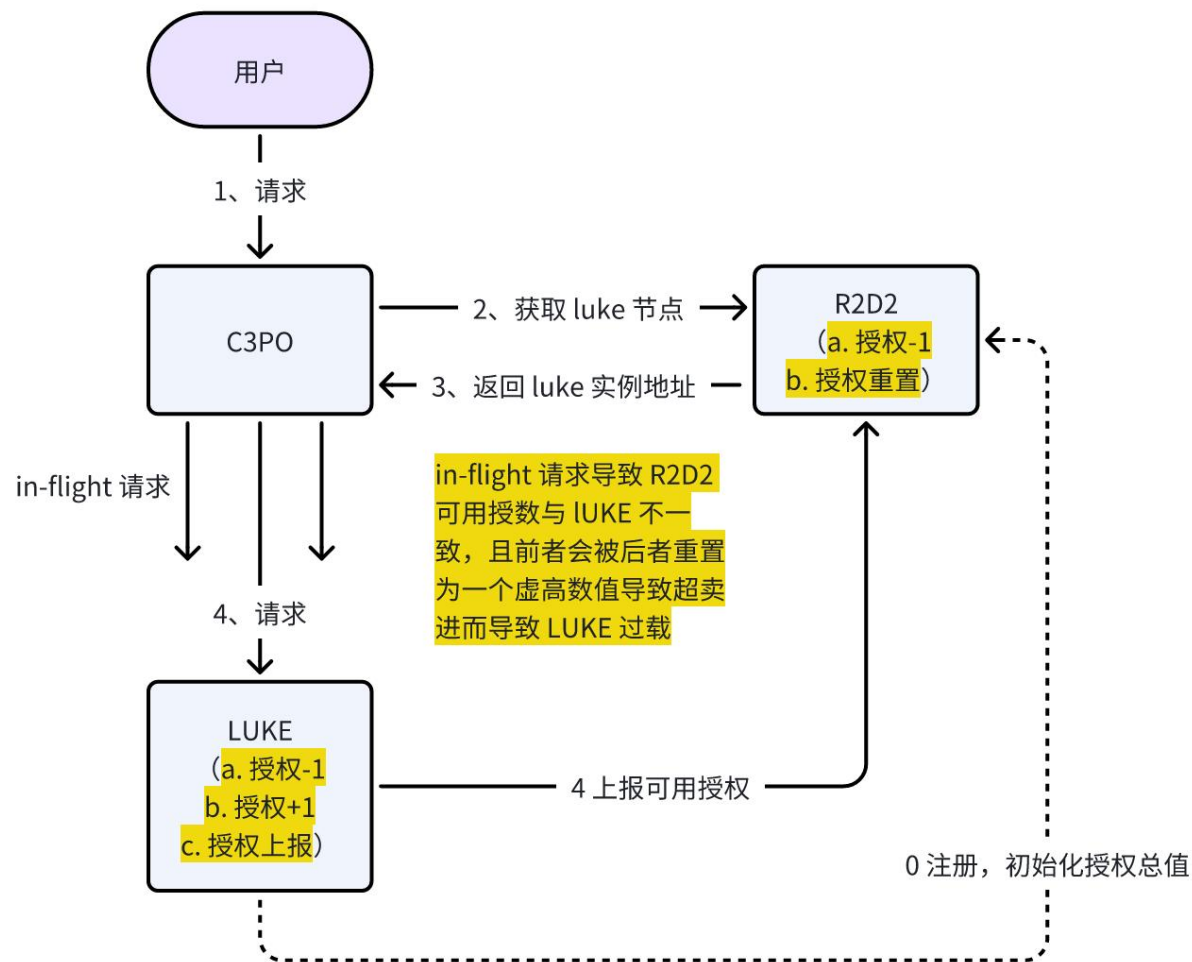


分布式负载精准控制问题分析

并发修改：两个节点同时修改授权状态且不互通

异常处理：消息乱序、延迟、丢失不可避免

数据更新：如何定义数据新旧，并在并发时正确合并

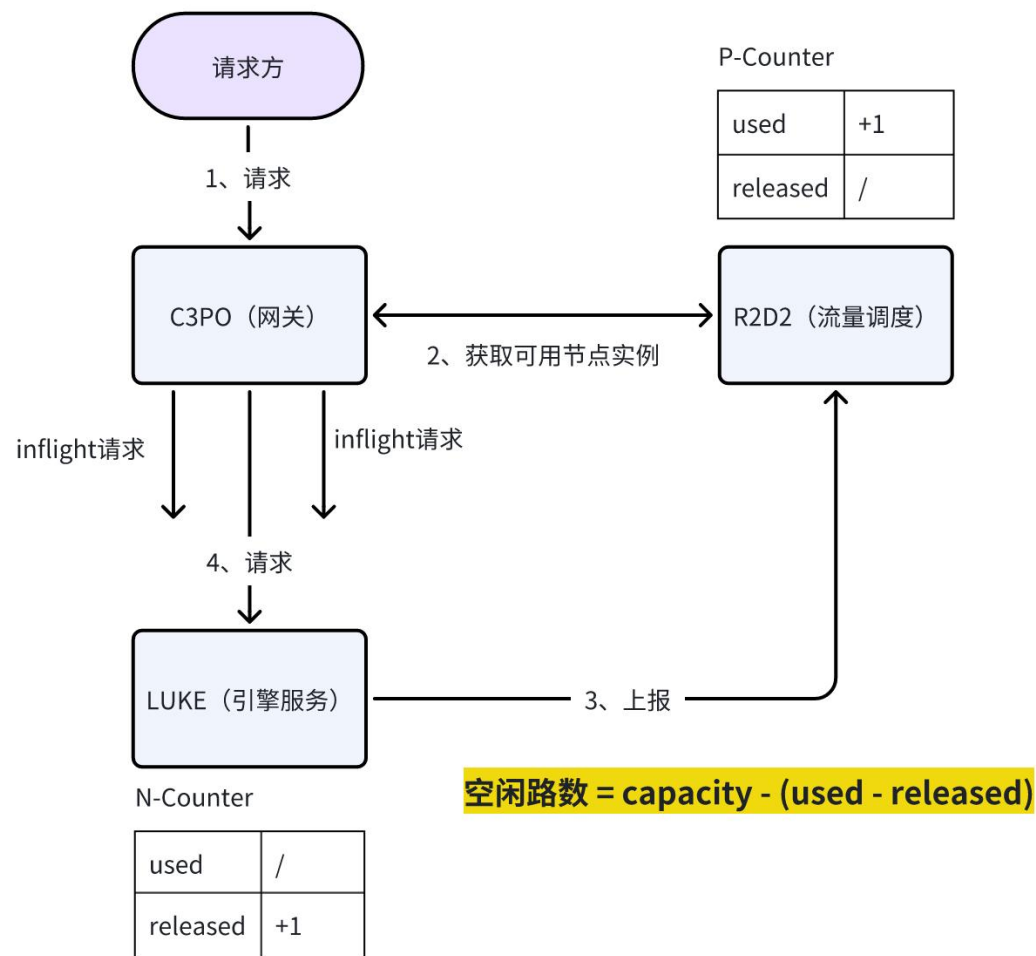


CRDT解决方案

基于 CRDT (Conflict-Free Replicated Data Type, 无冲突复制数据类型) PN-Counter 的授权并发冲突解决方案：不要求即时一致性，确保最终一致性，并通过数学性质保证冲突自动解决。

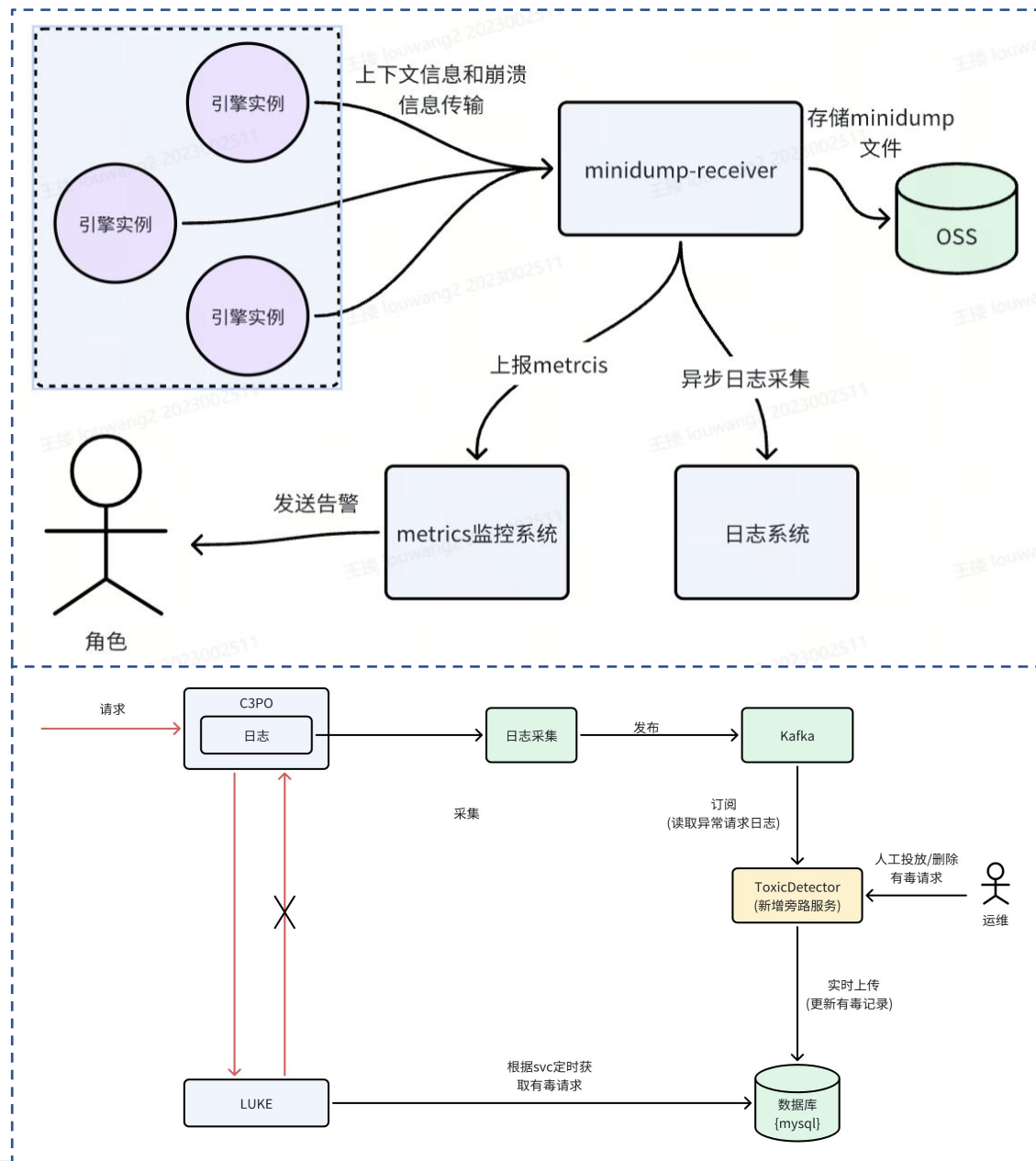
核心原则是：

- **无需协调**：节点可以独立修改数据，无需实时协调
- **确定性合并**：无论合并顺序如何，最终结果都相同
- **单调性**：系统状态只会向一个确定的方向发展
- **单侧补偿**：单侧可以感知数据乱序、丢失，独自确定补偿规则



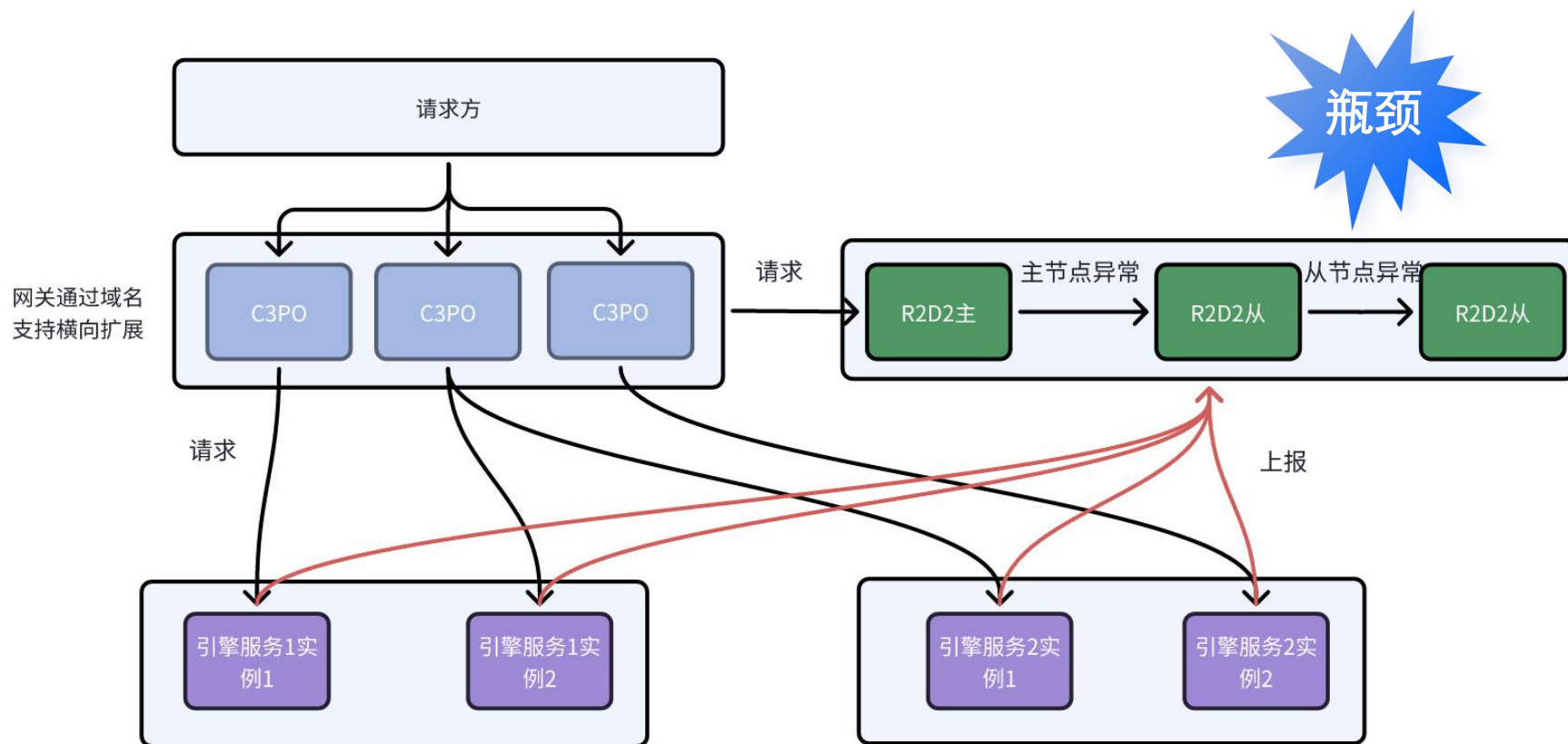
高效崩溃信息收集处理

- 传统core文件过大（数G大小）怎么办？
使用google breakpad轻量级方案，一个dump文件只有数百K
- 崩溃现场难保存，崩溃文件获取困难怎么办？
统一收集minidump并转存中间存储，自动化分析
- 崩溃复现困难怎么办？
崩溃时同步收集系统状态，进程状态，请求上下文信息，一次性保存所有辅助排查信息
- 引擎崩溃了无法即使感知怎么办？
打通与日志和告警系统的链路，崩溃自动触发告警，自动发送分析结果到对应责任人
- 类似请求持续进入导致连续崩溃怎么办？
实现有毒请求过滤功能，根据特征值过滤之前导致过崩溃的请求类似的请求

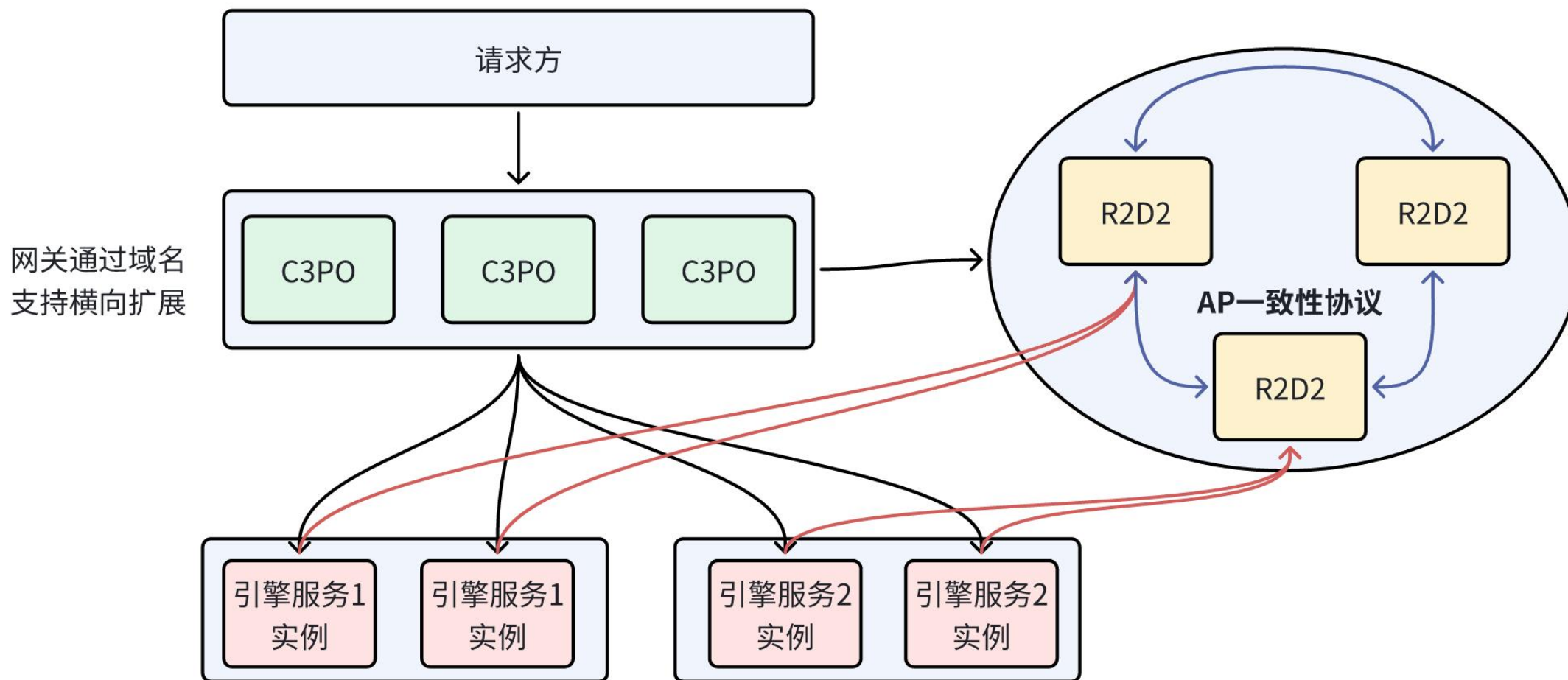


复杂流量调度带来的系统扩展挑战

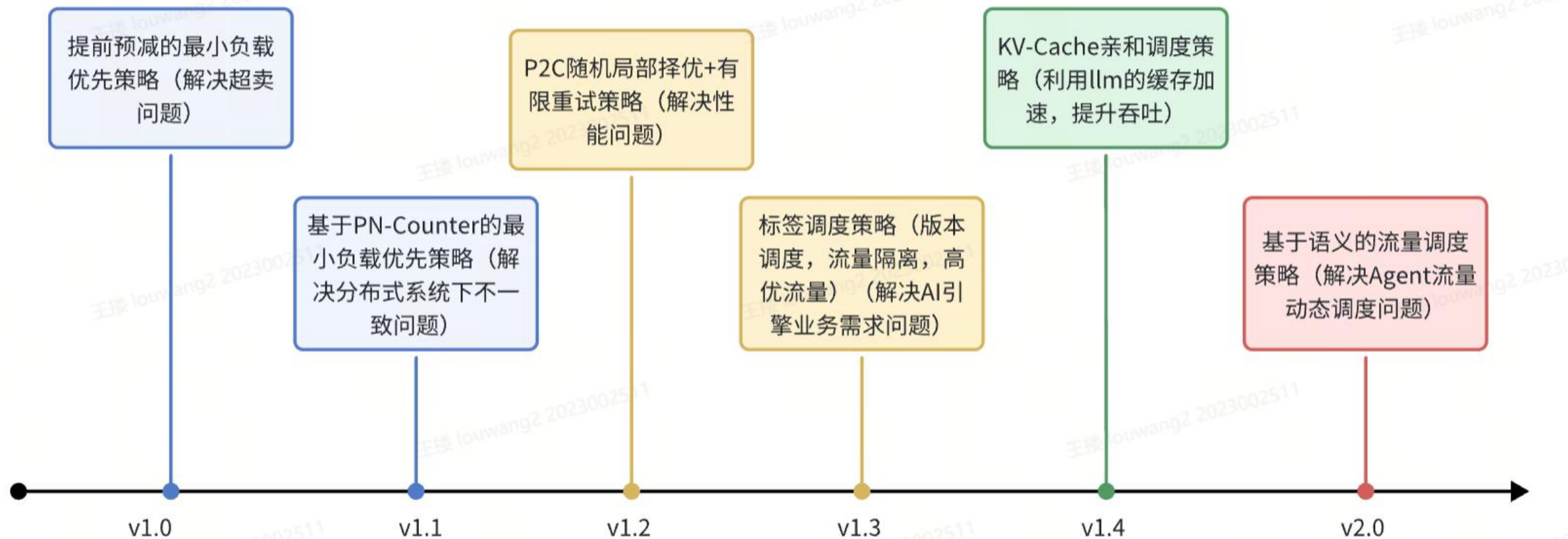
- 单请求必须在1ms以内处理完
- 请求扇出情况严重，百倍放大
- 单机性能要求极高（10w级qps），系统性能要求达百万级
- 引擎实例状态变化在毫秒级感知



WitMesh: 面向分布式AI引擎的AP一致性协议



■ 负载均衡策略演进经验分享



小结



统一框架、统一交互
流程、统一协议

管得住



负载感知、授权限
流、空闲调度

调得动



崩溃现场收集、链路
追踪、自动分析归集

追得回



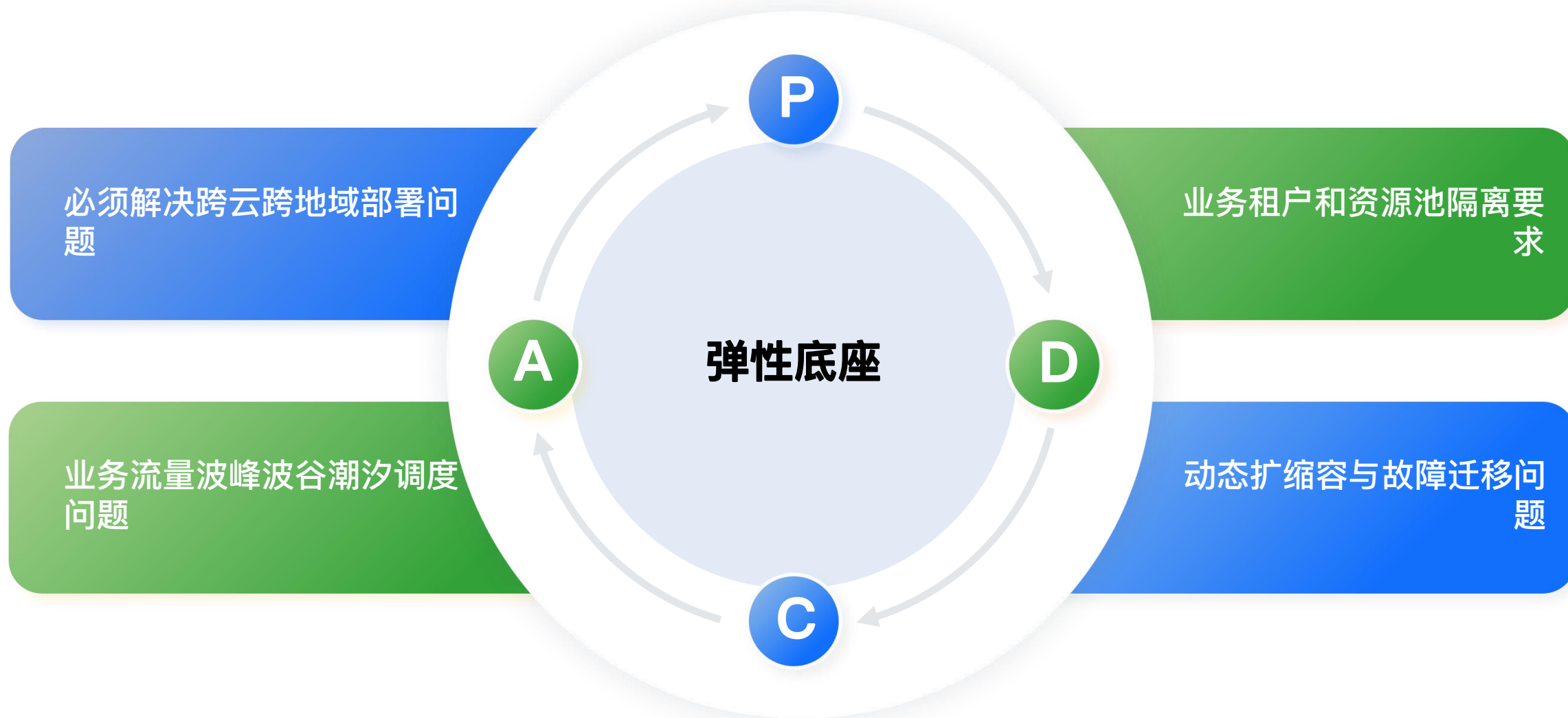
自组织集群、低延时
流量调度

扩得开

大模型时代的弹性底座

突破单云容量边界的跨云调度体系

■ 突破单云边界的多云弹性



■ 高速分发与高效加载

1

模型资源、引擎资源、服务镜像资源分层降低重复传输

2

多地域模型和镜像多仓库管理以及提前分发解决模型跨地域传输难题

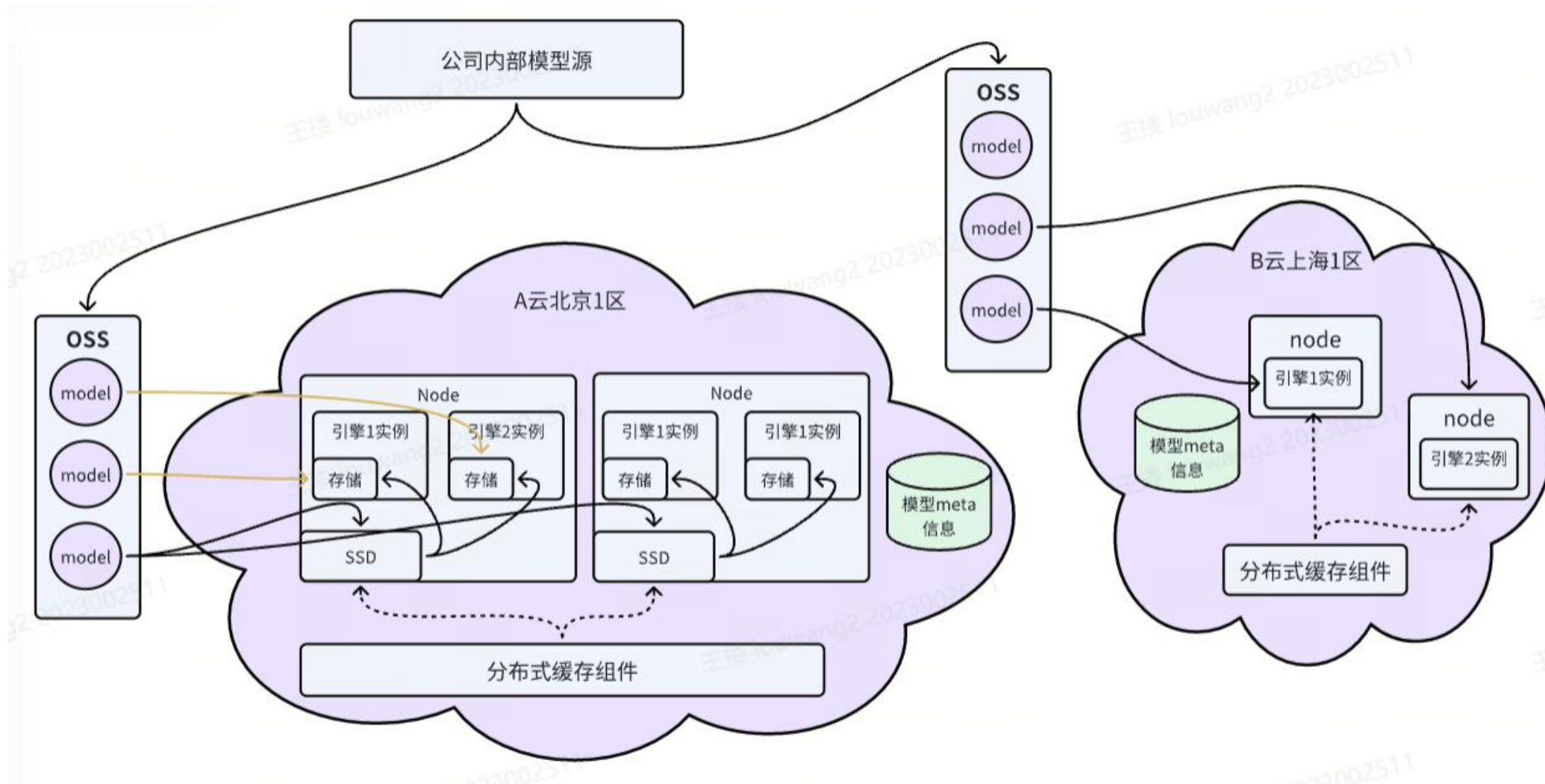
3

P2P高速分发大幅提升同地域模型和镜像分发速度，较传统集中式分发下载提速数十倍

4

集群内分布式缓存预热解决模型冷启动最后一公里

■ 多层分发框架



全方位感知与运维

全方位感知

- ✓ 监控：基于Prometheus的全方位系统指标监控
- ✓ 链路追踪：基于OpenTelemetry和Jaeger建立系统全链路追踪系统
- ✓ 日志：基于公有云方案和loggie开源方案的高速日志采集和查询系统
- ✓ 告警：分钟级甚至秒级指标异常告警系统

运维三板斧

- 重启：恢复异常实例
- 迁移：绕开异常节点
- 扩容：承接突发流量

智能化运维

哨兵系统-AIOps



根据设定条件动态生成报表

动态报表



线上问题分析自动化

问题分析



流量趋势预测智能化

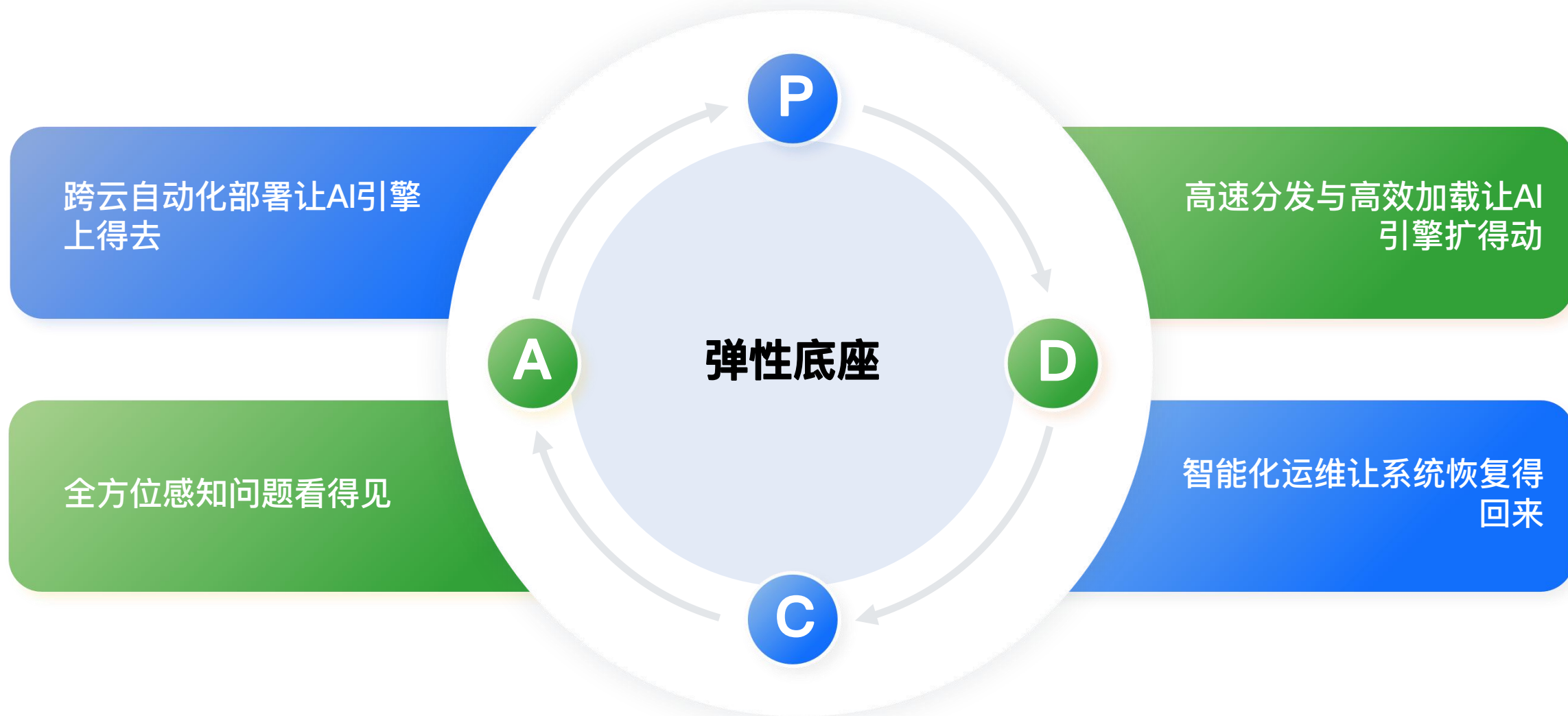
趋势预测



系统风险提前感知与消除

态势感知

小结



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 8月 📍 深圳 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海 🧑 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京 🧑 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

AiCon

全球人工智能开发与应用大会

深圳站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 8 月 21-22 日 / 深圳湾万丽酒店



参会咨询

专题: Harness Engineering: 模型之外的智能体工程

专题出品人: 柴思远

智谱 / 副总裁



专题: 超级个体与蜂群智能的共生进化

专题出品人: 张昊阳

EvoMap / 创始人



专题: AI Agent 高价值商业场景实战

专题出品人: 李响 博士

平安科技 / 智能两核 AI 部总经理



专题: 超级 App 的 AI 原生化: 端智能、端云协同与用户体验重构

专题出品人: 黎三平

京东 / 端技术负责人



专题: AI 原生新范式: Coding Agent 重构软件研发全流程

专题出品人: 林香鑫

网易游戏 / 高级技术经理



专题: 智能体时代的数据供给与治理

专题出品人: 陈廷梁 (王赟)

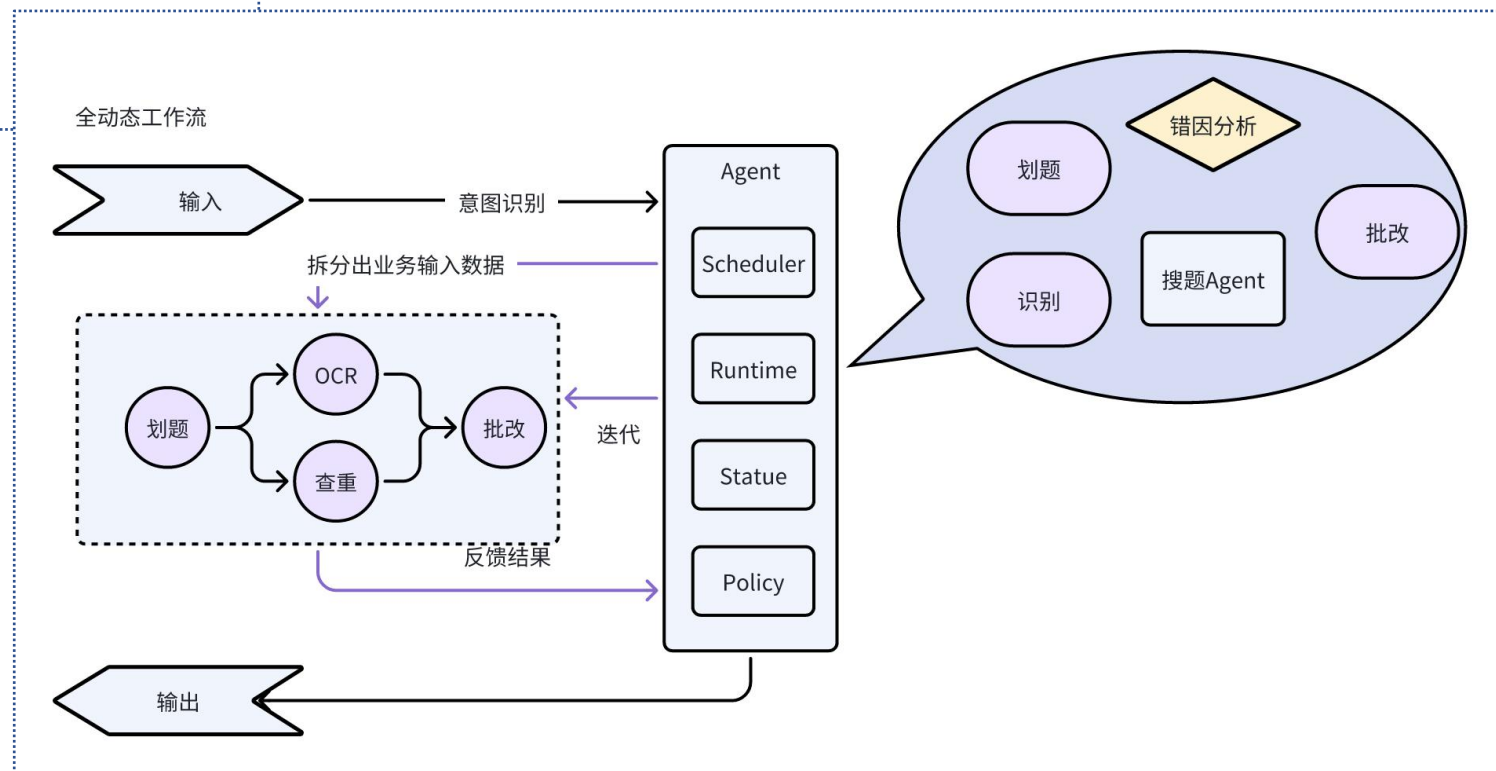
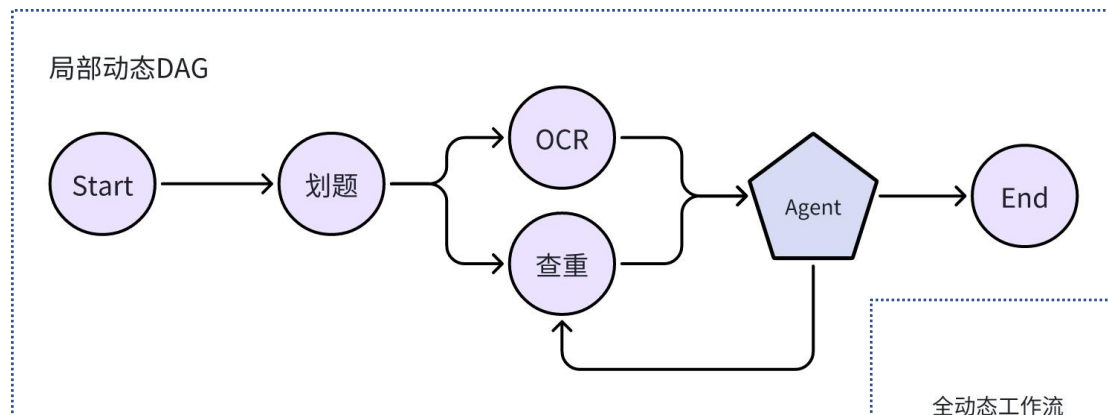
数新智能 / 创始人兼 CEO



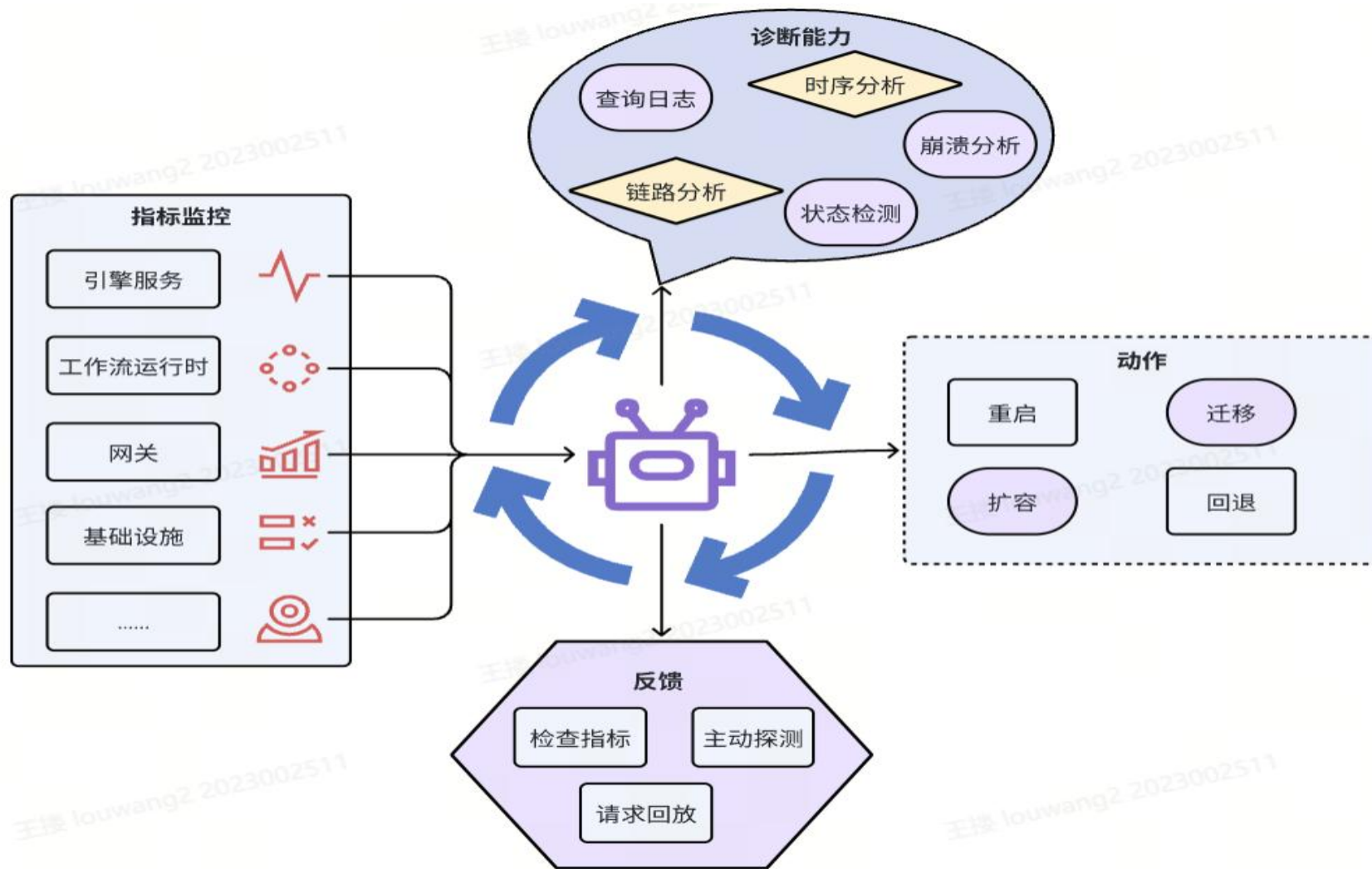
05 系统演进展望

从确定性的运行时底座走向动态的、可自治的自闭环系统

从“动静结合”到全动态的演进



为哨兵系统构建Loop Engineering



■ 最终 Takeaway

Agent 在教育超大规模生产环境中的落地，关键不只是模型效果，而是能否在可约束、可治理的Harness工程中，构建起支撑其 7× 24 小时稳定运行的工程底座。

- workflow编排：让AI复杂任务从静态流程执行，走向运行时决策。
- 引擎托管：让结合了AI特征和软件工程方法的引擎运行时沉淀成下一个“基础设施”。
- 基础设施底座：让Agent系统从效果可用，走向生产规模化稳定。

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会